

HASHDIT

HashDit

Smart Contract Security

Venus Hub NAV Deviation Sentinel EBrake

Security Audit Report

Venus

15 Oct 2026 - 16 Oct 2026

This report is provided on an "as is" basis and does not constitute investment advice, a recommendation, or an endorsement of the audited project. HashDit makes no representations or warranties regarding the safety, functionality, or correctness of the smart contracts beyond the scope defined herein.

The audit was performed based on the source code provided at the time of the engagement. Any subsequent modifications to the codebase may invalidate the findings of this report. HashDit shall not be held liable for any losses or damages resulting from the use, deployment, or interaction with the audited contracts.

The scope of this audit is limited to the smart contracts listed in this report. It does not cover off-chain components, front-end interfaces, or third-party integrations unless explicitly stated. Users and developers should conduct their own due diligence before interacting with or deploying the audited contracts.

Audit Period	15 Oct 2026 - 16 Oct 2026
Overall Risk	**Medium**
Project Name	Venus Hub NAV Deviation Sentinel EBrake Audit
Github	https://github.com/VenusProtocol/venus-periphery/pull/75
Commit	ecf8d8f93a440110fa8cd8f23b908cd5ef454004

Smart Contract List

No.	Contract Name	Verdict	Details
1	HubNavDeviationSentinel.sol	Medium	[M01]-Fixed [L01]-Fixed [L02]-Fixed
2	EBrake.sol	Passed	-
3	IEBrake.sol	Passed	-
4	IHubLiquidity.sol	Passed	-

[M01] HubNavDeviationSentinel pause materiality gate divides by a live, same-block-movable IHub.totalAssets(), so Hub flow sets the brake's firing threshold	
Contract	HubNavDeviationSentinel.sol
Severity	**Medium**

Description	The _exceedsMinHubNavGap (HubNavDeviationSentinel.sol:467-468) evaluates
	
	JavaScript
	`gap > IHub(hub).totalAssets() * minHubNavGapBps / MAX_DEVIATION_BPS`
	
	against a live read taken inside the same call that decides the pause, reached from both the ordinary threshold branch (:457) and the closed-band branch (:440). There is no snapshot, no TWAP, no persistence counter, no cooldown and no committed observation anywhere on the path; the whole verdict is recomputed per call (:331/:381). Hub NAV is the quantity every Hub depositor and redeemer moves, so the materiality floor governance believes it set through minHubNavGapBps (:88, :220) is in fact set per block by whoever last moved Hub flow.
	
	An armed pair (config.enabled true, :397) with minHubNavGapBps non-zero, which it is by default (initialize writes 100 = 1% of Hub NAV at :220).
	
	FABRICATION: a real deviation past pauseDownBps/pauseUpBps whose gap sits below the current floor, plus an ordinary Hub redemption while the keeper's monitor samples the public view - cost is gas, and the actor exits at fair value.
	
	SUPPRESSION: a Breached condition plus capital of about 100*gap - NAV that persists across the keeper's block, frontrunning the keeper's public handleNavGuardDeviation transaction.
	
	Hub deposit and redemption are permissionless (the Hub is a standard ERC-4626), so neither direction needs the trusted-keeper role or any privilege. Explicitly NOT a flash-loan path: the read happens inside the keeper's own transaction, so suppressing capital must be exposed across that block to the very pause it suppresses.
	
	JavaScript
	`function handleNavGuardDeviation(address yieldGroup, address resource) external onlyKeeper {  // @audit evidence`
	`(`
	`NavGuardCheckStatus status,`
	`address hub,`
	`uint256 observedValue,`
	`uint256 centre,`

```
            uint256 minAllowedValue,
            uint256 maxAllowedValue
        ) = checkNavGuardDeviation(yieldGroup, resource);  // @audit
evidence

        if (status == NavGuardCheckStatus.MonitoringDisabled) revert
NavGuardDisabled(yieldGroup, resource);
        if (status == NavGuardCheckStatus.YieldGroupNotRegistered)
revert YieldGroupNotRegistered(hub, yieldGroup);
        if (status == NavGuardCheckStatus.ResourceNotRegistered) revert
ResourceNotRegistered(yieldGroup, resource);
        if (status == NavGuardCheckStatus.ObservedValueZero) revert
NavGuardObservedValueZero(resource);
        if (status == NavGuardCheckStatus.CentreZero) revert
NavGuardCentreZero(resource);
        if (status == NavGuardCheckStatus.WithinThreshold) revert
DeviationWithinThreshold(resource, observedValue);  // @audit evidence

        // Backstop for a status the list above does not name: fall
through to doing nothing rather
        // than to pausing a Hub on a verdict this function was never
taught to read.
        if (status != NavGuardCheckStatus.Breached) return;

        EBRAKE.pauseHub(hub);  // @audit evidence

        emit NavGuardDeviationHandled(  // @audit evidence
            hub,
            yieldGroup,
            resource,
            observedValue,
            centre,
            minAllowedValue,
            maxAllowedValue
        );
    }
// ...
    function checkNavGuardDeviation(  // @audit evidence
        address yieldGroup,
        address resource
    )
        public
        view
        returns (
            NavGuardCheckStatus status,
            address hub,
            uint256 observedValue,
            uint256 centre,
            uint256 minAllowedValue,
            uint256 maxAllowedValue
        )
```

```
    {
        NavGuardConfig memory config =
navGuardConfigs[yieldGroup][resource];
        if (!config.enabled) return
(NavGuardCheckStatus.MonitoringDisabled, address(0), 0, 0, 0, 0);  //
@audit evidence

        hub = config.hub;

        if (!IHub(hub).yieldGroupConfig(yieldGroup).registered) {
            return (NavGuardCheckStatus.YieldGroupNotRegistered, hub, 0,
0, 0, 0);
        }

        (bool registered, , ) =
IYieldGroupNav(yieldGroup).resourceConfig(resource);
        if (!registered) return
(NavGuardCheckStatus.ResourceNotRegistered, hub, 0, 0, 0, 0);

        (observedValue, minAllowedValue, maxAllowedValue, , ) =
IYieldGroupNav(yieldGroup).navGuardStatus(resource);

        // Our pause thresholds are measured from the centre, not the
Hub's min and max, because those
        // include the Hub's own gap and would shift our thresholds
whenever that gap is changed.
        IYieldGroupNav.NavBand memory band =
IYieldGroupNav(yieldGroup).navGuard(resource);

        // The centre grows by `driftBps` a year but is only saved on a
deposit, redeem or re-anchor,
        // so add the growth since `driftFrom`, the same way the Hub's
`NavGuard._grown` does.
        uint256 drift = (uint256(band.centre) * band.driftBps *
(block.timestamp - band.driftFrom)) /
            (uint256(MAX_DEVIATION_BPS) * 365 days);
        centre = band.centre + drift;  // @audit evidence

        // Two zero cases are handled first, as the percentage check
below would pause on either:
        // - `observedValue` is 0 when unreadable or priced at nothing,
which is below every threshold.
        // - `centre` is 0 after a full exit closes the band, which
makes every threshold 0.
        if (observedValue == 0) {
            return (
                NavGuardCheckStatus.ObservedValueZero,
                hub,
                observedValue,
                centre,
                minAllowedValue,
```

```
                maxAllowedValue
            );
        }
        if (centre == 0) {
            // The band is closed but the position still reports a
value. With the cap on, the Hub
            // values it at 0, so Hub NAV is short by `observedValue`.
That pauses only if the
            // downside is watched and the shortfall is over
`minHubNavGapBps` of Hub NAV, so dust
            // cannot freeze the Hub.
            bool closedBandBreached = band.capEnabled &&  // @audit
evidence
                config.pauseDownBps != 0 &&
                _exceedsMinHubNavGap(hub, observedValue);  // @audit
evidence

            status = closedBandBreached ? NavGuardCheckStatus.Breached :
NavGuardCheckStatus.CentreZero;  // @audit evidence
            return (status, hub, observedValue, centre, minAllowedValue,
maxAllowedValue);
        }

        uint256 downTripPoint = (centre * (MAX_DEVIATION_BPS -
config.pauseDownBps)) / MAX_DEVIATION_BPS;  // @audit evidence
        uint256 upTripPoint = (centre * (MAX_DEVIATION_BPS +
config.pauseUpBps)) / MAX_DEVIATION_BPS;

        // A threshold of 0 means that side is off, not that it pauses
on any move.
        bool breachedDown = config.pauseDownBps != 0 && observedValue <
downTripPoint;  // @audit evidence
        bool breachedUp = config.pauseUpBps != 0 && observedValue >
upTripPoint;  // @audit evidence

        // Crossing a threshold is not enough: the gap must also be over
`minHubNavGapBps` of the
        // Hub's NAV, the same test a closed band gets. The side that
crossed puts `observedValue`
        // above or below the centre, so the subtraction cannot
underflow.
        bool breached = (breachedDown || breachedUp) &&  // @audit
evidence
            _exceedsMinHubNavGap(hub, breachedUp ? observedValue -
centre : centre - observedValue);  // @audit evidence

        status = breached ? NavGuardCheckStatus.Breached :
NavGuardCheckStatus.WithinThreshold;  // @audit evidence
    }
// ...
```

```
    function _exceedsMinHubNavGap(address hub, uint256 gap) private view
returns (bool) {  // @audit evidence
        return gap > (IHub(hub).totalAssets() * minHubNavGapBps) /
MAX_DEVIATION_BPS;  // @audit evidence
    }
```

An unprivileged third party, not governance, decides what counts as material. Fabrication drives EBRAKE.pauseHub (:351) and freezes every deposit and redemption on the whole Hub - recoverable only by a governance VIP (:15) - while the fabricating redeemer has already exited at fair value and the holders who stayed are locked in.

Suppression silently disables the circuit breaker at the moment it is due, leaving the Hub pricing shares off a NAV the sentinel has already judged materially wrong, so redeemers extract the overstatement from the remaining holders, with no event emitted on the declined escalation.

Recommendation

Harden the materiality scale by replacing the single live totalAssets() read with a median or TWAP over a spacing-enforced window — never min or max, each of which is blind to one direction of the attack — accepting that a longer window buys verdict integrity at the cost of a wider window for informed holders to exit before the pause lands.

Alternatively, replace the Hub-NAV-relative gate with a fixed per-resource minimum gap (e.g. NavGuardConfig.minGapAssets) set by governance. It still prevents dust and small positions from freezing the Hub, cannot be moved by deposits, redemptions or donations, and removes the sentinel's dependency on Hub.totalAssets(). A median/TWAP raises the cost of suppression but does not remove it, since the capital in this attack is already held across blocks, and it adds stored observations and a writer role.

```javascript
function _exceedsMinGap(NavGuardConfig memory config, uint256 gap)
private pure returns (bool) {
    return gap > config.minGapAssets;
}
```

Status

Fixed in commit [29b5e78](). Replaced the Hub-NAV-relative gate with a fixed per-resource minPauseGap, which is rejected at zero. Also removed minHubNavGapBps, setMinHubNavGapBps and _exceedsMinHubNavGap. The pause decision no longer reads Hub.totalAssets() and Hub deposits and redemptions cannot move it:

```javascript
if (minPauseGap == 0) revert ZeroMinPauseGap();
...
bool breached = (breachedDown || breachedUp) &&
    (breachedUp ? observedValue - centre : centre - observedValue) >
config.minPauseGap;
```

[L01] Pause trip points are sized off the band centre, so inflows dilute an ongoing breach	
Contract	HubNavDeviationSentinel.sol
Severity	Low

Description

checkNavGuardDeviation sizes both trip points as a percentage of the band's centre:

```javascript
uint256 downTripPoint = (centre * (MAX_DEVIATION_BPS -
config.pauseDownBps)) / MAX_DEVIATION_BPS;
uint256 upTripPoint = (centre * (MAX_DEVIATION_BPS +
config.pauseUpBps)) / MAX_DEVIATION_BPS;
The Hub's own band (NavGuard._currentBand) is centred on centre but
its width is sized off anchor:
floor = centre - (anchor * band.downGapBps) / BPS;
cap = centre + (anchor * band.upGapBps) / BPS;
```

Every deposit or reallocation routed into the resource goes through YieldGroupCentrifuge._dispatchDeposit -> NavGuard._recordInflow, which adds the inflow to centre but leaves anchor unchanged. AdapterCentrifuge counts pendingDepositRequest at face value, so the observed value rises by the same amount. The gap between observed and centre, and therefore the amount by which the Hub mis-prices the resource, stay exactly the same; only the gap as a percentage of centre shrinks, so the sentinel stops tripping while the mispricing persists.

A variant affects closed bands: when a band is closed (centre = 0) with a residual value and the cap enabled, the Hub values the residual at 0. The next inflow reopens the band with anchor = 0 (a zero-width band); the Hub keeps valuing the residual at 0, but the sentinel leaves the closed-band branch and the percentage test is diluted by the new inflow.

Example

Setup: Hub NAV 4,000,000; a Centrifuge resource with centre = anchor = 1,000,000; Hub gaps ±2% (floor 980,000, cap 1,020,000, both enabled); sentinel pauseDownBps 8% (trip point 920,000) and minHubNavGapBps 1% (40,000).

1. Centrifuge marks the position at 850,000. The Hub values it at the floor 980,000, overstating it by 130,000. The sentinel sees 850,000 < 920,000 and a gap of 150,000 > 40,000, so the verdict is Breached.
2. Before the keeper acts, a user deposit of 1,000,000 is routed into the resource. centre becomes 2,000,000 and the observed value 1,850,000, while anchor stays 1,000,000. The floor moves to 1,980,000, so the Hub still overstates the position by 130,000.
3. The keeper calls handleNavGuardDeviation. The trip point is now 2,000,000 × 92% = 1,840,000 and the observed value 1,850,000 is above it, so the verdict is WithinThreshold and the call reverts with DeviationWithinThreshold. The Hub is not paused.
4. The Hub keeps pricing shares with a 130,000 overstatement (~2.6% of NAV). Redeemers exit at the inflated price and the remaining holders bear the loss, while every keeper check keeps returning WithinThreshold. With

	an anchor-based trip point (2,000,000 − 1,000,000 × 8% = 1,920,000) the breach would still be caught. **Impact** A material mispricing stays unpaused, triggered by ordinary flows without any attacker. It is rated Low because it requires a prior counterparty misreport and sufficient inflow before the keeper acts, and the Hub's floor/cap still bound the error.

Recommendation	Size the trip points off the drift-grown anchor, i.e. the same base the Hub uses for its band width, so that inflows only move the centre and cannot widen the trip range:
	JavaScript
	`uint256 elapsed = block.timestamp - band.driftFrom;`
	`uint256 anchor = band.anchor + (uint256(band.anchor) * band.driftBps * elapsed) / (uint256(MAX_DEVIATION_BPS) * 365 days);`
	`uint256 downTripPoint = centre - (anchor * config.pauseDownBps) / MAX_DEVIATION_BPS;`
	`uint256 upTripPoint = centre + (anchor * config.pauseUpBps) / MAX_DEVIATION_BPS;`
	With this change the example above trips at 1,920,000 instead of 1,840,000 and the breach is caught. When anchor is 0 (reopened band or fresh re-entry) both trip points equal centre and only minHubNavGapBps gates the pause; document this. Add a test where a breach is followed by an inflow into the resource and must still return Breached.

Status	Fixed in commit 3ba7461. The trip points are now sized off the drift-grown anchor, which is the same base the Hub uses for its band width. An inflow into the resource can therefore no longer dilute an ongoing breach:
	JavaScript
	`uint256 anchor = _driftGrown(band.anchor, band);`
	`uint256 downTripPoint = centre - (anchor * config.pauseDownBps) / MAX_DEVIATION_BPS;`
	`uint256 upTripPoint = centre + (anchor * config.pauseUpBps) / MAX_DEVIATION_BPS;`

[L02] An open band whose counterparty reads back zero is never paused, while the Hub prices it at zero	
Contract	HubNavDeviationSentinel.sol
Severity	Low

Description

checkNavGuardDeviation returns a no-action status whenever the observed value is zero:

```javascript
if (observedValue == 0) {
    return (
        NavGuardCheckStatus.ObservedValueZero,
        hub, observedValue, centre, minAllowedValue,
maxAllowedValue
    );
}
```

On the Hub side, NavGuard._guardedNav passes a zero straight through (the floor does not apply), so the Hub immediately removes the whole position from its NAV while deposits and redemptions continue:

```javascript
if (observedNav == 0) return 0;
```

Venus's own full exit always closes the band (_recordOutflow sets centre to 0), and the Hub itself treats a counterparty-published zero as untrustworthy (_updateNavGuard: "a zero the counterparty publishes must not close the band"). A state with centre > 0 and observed == 0 can therefore only originate from the counterparty. AdapterCentrifuge returns a readable 0 without reverting when both its face and share components are 0, e.g. when the whole position sits in manager-tracked buckets (a pending deposit or a full pending redemption) and the vault's baseManager (upgradeable and outside Venus control) stops returning the position, or when the issuer force-transfers all shares. The next inflow then hits _recordInflow's value == amount reset (anchor = 0, centre = amount), erasing the previous centre.

The behaviour is documented as intentional, but the stated rationale does not hold: an unreadable value already reverts through Hub.totalAssets() in _exceedsMinHubNavGap, so it cannot be mistaken for a breach; the Hub refusing to close a band on a published zero argues for pausing rather than against it; and a governance write-off (forceRemoveResource) requires the Hub to be paused already.

Impact

While the band is still open, the Hub prices the whole position at zero and keeps accepting deposits and redemptions, but the sentinel never pauses. Depositors can buy shares at the depressed price and capture value from existing holders once the position reappears, and any inflow routed into the resource in the meantime resets the band and erases the record of the previous centre. Rated

	Low because the trigger is on the counterparty side and rare, but the mispricing covers the full position and can be arbitraged.

Recommendation	Treat a zero observation on an open band as a full downside breach, still subject to the materiality check:
	
	JavaScript
	`if (observedValue == 0) {`
	`  if (centre != 0 && config.pauseDownBps != 0 && _exceedsMinHubNavGap(hub, centre)) {`
	`  return (NavGuardCheckStatus.Breached, hub, observedValue, centre, minAllowedValue, maxAllowedValue);`
	`  }`
	`  return (NavGuardCheckStatus.ObservedValueZero, hub, observedValue, centre, minAllowedValue, maxAllowedValue);`
	`}`
	
	This introduces no false positives: an unreadable value still reverts through Hub.totalAssets(), a Venus-initiated full exit has centre = 0, and an unconfigured band has centre = 0.
	Optional hardening (changes the current design of not pausing on a price outage; project's decision): wrap the totalAssets() read in _exceedsMinHubNavGap in try/catch and treat an unreadable NAV as material once the resource's own threshold has been crossed, so the Hub does not reopen unprotected when a feed recovers. This only takes effect together with the fix above, because the zero branch returns before the NAV is read. Update the ObservedValueZero / NavGuardObservedValueZero comments and add a test for this case.
	
	Note on interaction with [M01]: the no-false-positive argument above relies on _exceedsMinHubNavGap reading Hub.totalAssets() live, so an unreadable position still reverts. If the materiality scale is changed to a stored median/TWAP or a fixed per-resource amount as recommended in [M01], an unreadable position will no longer revert at that point and this fix will also pause the Hub on a price outage. This matches the optional hardening above; the project should confirm it is the intended behaviour when applying both fixes.

Status	Fixed in commit 02318b3. A zero reading on an open band is now treated as a full downside loss. It pauses the Hub when the downside is watched and the centre exceeds minPauseGap. An unreadable price arrives as the same zero, so a price outage now pauses the Hub as well:
	
	JavaScript
	`bool zeroReadingBreached = centre != 0 && config.pauseDownBps != 0 && centre > config.minPauseGap;`
	`status = zeroReadingBreached ? NavGuardCheckStatus.Breached : NavGuardCheckStatus.ObservedValueZero;`